VERIQTA

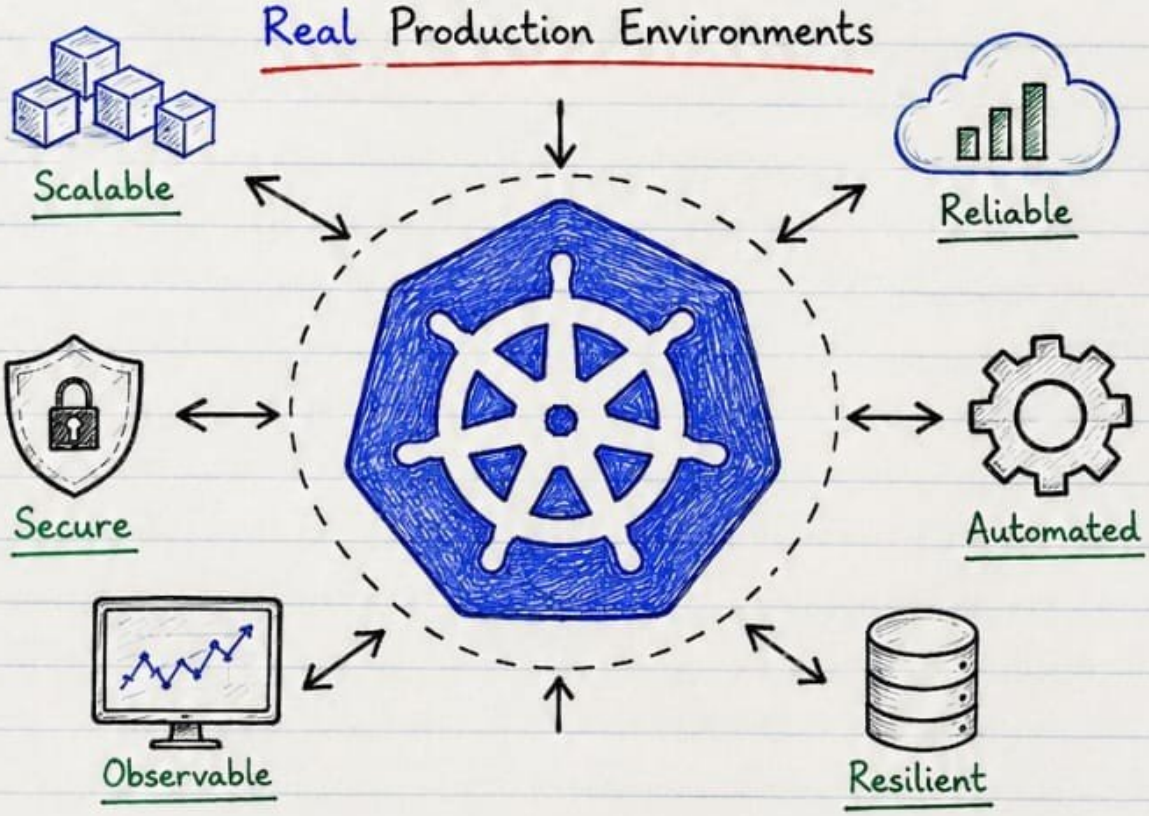
@ veriqta

KUBERNETES

PRODUCTION ENGINEERING

VOLUME 2

Running Kubernetes in
Real Production Environments



- ✓ Production Patterns
- ✓ Real-World Scenarios
- ✓ Best Practices
- ✓ Troubleshooting
- ✓ Performance
- ✓ Operational Excellence

"☆ Build it right. Run it strong. Keep it reliable."

VERIQTA @ veriqta

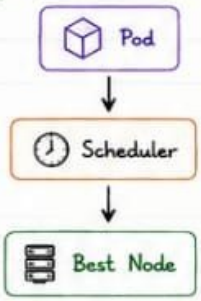
VOLUME 2 KUBERNETES PRODUCTION ENGINEERING

1. KUBERNETES SCHEDULING

"The Scheduler puts the right Pod on the right Node at the right time."

1 WHAT IS KUBERNETES SCHEDULING?

- Scheduling is the process of assigning a Pod to the most suitable Node.
- The Kubernetes Scheduler is a control plane component.
- It considers many factors to make the best possible decision.
- Goal: Run Pods efficiently, reliably and in the right place.

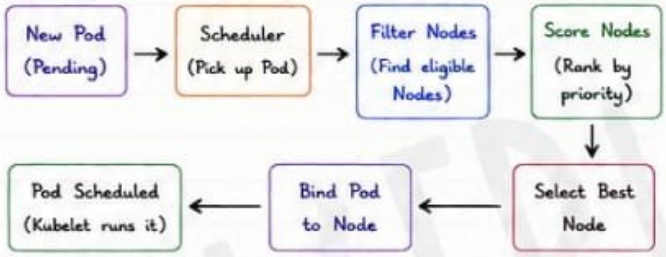


Match Pod requirements with the best available Node.

2 SCHEDULING PROCESS (HIGH LEVEL)

1. User creates a Pod (without nodeName).
2. Pod is stored in etcd with status: Pending.
3. Scheduler watches for unscheduled Pods.
4. Scheduler finds suitable Nodes.
5. Scheduler binds the Pod to the selected Node.
6. Kubelet on that Node starts the Pod.
7. Pod status becomes: Running.

3 SCHEDULER WORKFLOW



Think: Filter removes bad choices. Scoring ranks the good choices.

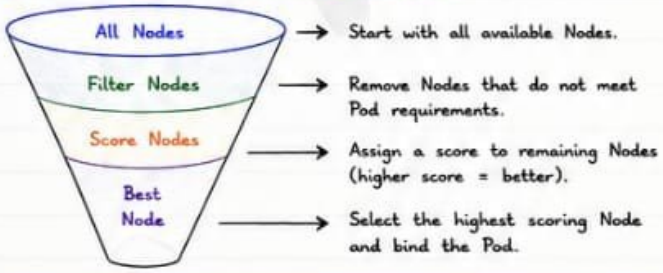
4 SCHEDULING DECISIONS

- **Resource Fit**
  Does the Node have enough CPU, Memory and other resources?
- **Policy Fit**
  Does the Pod satisfy rules like affinity, taints/tolerations, etc.?
- **Performance**
  Which Node gives the best balance and efficiency?
- **Constraints**
  Respect Node conditions, limits and cluster policies.

DECISION FACTORS

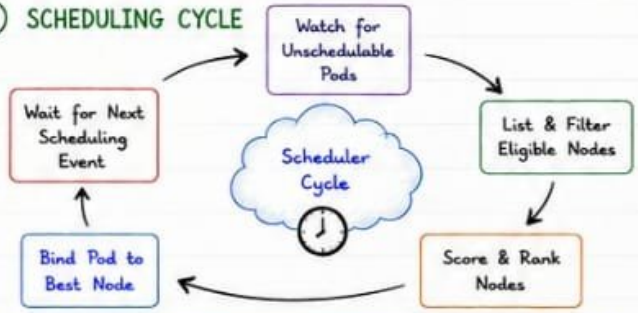
- ✓ CPU / Memory
- ✓ Node Capacity
- ✓ Taints & Tolerations
- ✓ Node Affinity
- ✓ Pod Affinity / Anti-Affinity
- ✓ Topology Spread
- ✓ Node Conditions
- ✓ Custom Policies
- ✓ Inter-Pod Constraints

5 NODE SELECTION (HOW IT WORKS)



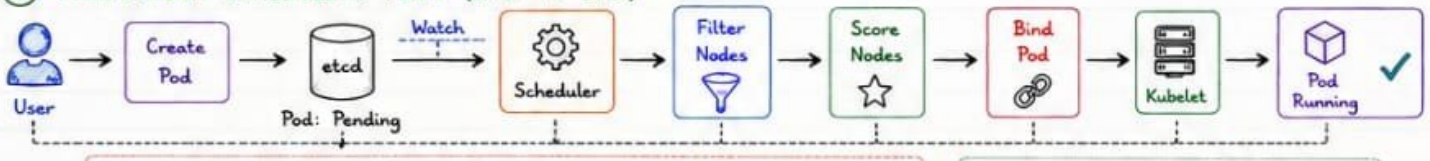
Key: Filtering first, scoring next, selection last.

6 SCHEDULING CYCLE



The cycle repeats continuously for new Pods or when cluster state changes.

7 PRODUCTION SCHEDULING FLOW (END-TO-END)



CONTROL PLANE (Decision making) → WORKER NODE (Execution)

KEY TAKEAWAYS

- ✓ Scheduler watches for new Pods.
- ✓ It filters, scores and selects the best Node.
- ✓ The right scheduling = performance + reliability.
- ✓ Good scheduling = happy Pods & happy cluster!

COMMON SCHEDULING EVENTS

- New Pod created
- Node added / removed
- Node capacity changed
- Pod updated (affinity, resources)
- Taints / tolerations updated
- Node condition changed

BEST PRACTICES

- Set proper requests & limits.
- Use node affinity & labels.
- Use taints for dedicated workloads.
- Keep Nodes healthy.
- Monitor scheduling failures.
- Test with real workloads.

"Great scheduling is invisible when it works, and priceless when it doesn't."

VERIQTA @ veriqta

VOLUME 2 KUBERNETES PRODUCTION ENGINEERING

2. NODE SELECTORS & NODE AFFINITY

"Control where your Pods run by matching Node labels."

1 NODESELECTOR

- Simplest way to constrain Pods to nodes with specific labels.
- Exact match (AND) of key-value pairs.
- Hard requirement.

HOW IT WORKS

The scheduler looks for Nodes that have ALL the specified labels and values.

EXAMPLE

Pod Spec (using nodeSelector)

```
nodeSelector:
  disktype: ssd
  environment: production
```

4 NODE LABELS ON NODES

Node A	Node B	Node C
diskType: ssd	diskType: hdd	diskType: ssd
env: production	env: production	env: dev
zone: us-east-1a	zone: us-east-1b	zone: us-east-1a

Pod → ✓ Scheduled | X Not a match | X Not a match

KEY POINTS

- ✓ Good for simple use cases.
- ✓ Requires nodes to be labeled consistently.
- ✓ Not flexible for "OR" or complex rules.

Use Cases

- Dedicated hardware (GPU, SSD, High-Mem)
- Environment separation (dev/staging/prod)
- Compliance & isolation

2 NODE AFFINITY OVERVIEW

- More powerful & expressive than nodeSelector.
- Uses matchExpressions for flexible matching.
- Two types: Required (hard) & Preferred (soft).

TYPES OF NODE AFFINITY

REQUIRED AFFINITY (Hard Requirement)

Pod MUST be scheduled on Nodes that satisfy the rules.

If no Node matches, Pod will remain Pending.

PREFERRED AFFINITY (Soft Preference)

Scheduler tries to place Pod on matching Nodes, but can use others if needed.

Uses weight (1-100) to rank preferences.

3 REQUIRED AFFINITY (HARD)

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - nodeSelectorTerms:
          - matchExpressions:
              - key: disktype
                operator: In
                values: ["ssd"]
              - key: environment
                operator: In
                values: ["production"]
```

★ Pod will only run on Nodes with disktype=ssd AND environment=production

4 PREFERRED AFFINITY (SOFT)

```
affinity:
  nodeAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 80
        preference:
          matchExpressions:
            - key: topology.kubernetes.io/zone
              operator: In
              values: ["us-east-1a"]
      - weight: 20
        preference:
          matchExpressions:
            - key: disktype
              operator: In
              values: ["ssd"]
```

★ Scheduler prefers zone us-east-1a, then prefers disktype=ssd.

5 SCHEDULING RULES (CHEAT SHEET)

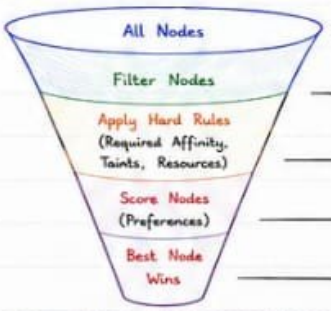
RULE TYPE	DESCRIPTION	USE WHEN...
nodeSelector	Exact match (AND) of key-value pairs	Simple, strict matching is enough
Required Affinity	All matchExpressions must match (AND)	You have strict requirements
Preferred Affinity	Scheduler prefers but not mandatory	You want preference but have fallback
Taints & Tolerations	Repels unless tolerated	Reserve Nodes for specific workloads
Multiple Rules	All hard rules must pass first, then preferences apply	Complex scheduling scenarios
Scoring	Scheduler scores Nodes and picks highest	Best fit based on resources & prefs

MATCH EXPRESSIONS

OPERATOR	MEANING
In	Key value is in the list
NotIn	Key value is NOT in the list
Exists	Key exists
DoesNotExist	Key does NOT exist
Gt	Greater than
Lt	Less than

Tip: Use Exists / In most often.

6 NODE SELECTION (HOW IT WORKS)



- All Nodes → Start with all available Nodes in the cluster.
- Filter Nodes → Remove Nodes that don't meet basic requirements (resources, taints, etc).
- Apply Hard Rules → Only Nodes that pass hard rules remain.
- Score Nodes → Score remaining Nodes based on preferences.
- Best Node Wins → Select the highest scoring Node.

★ Filtering first, scoring next, selection last.

7 PRODUCTION EXAMPLES

1. Run on GPU Nodes

```
nodeSelector:
  accelerator: nvidia
# or with affinity
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnored...
      nodeSelectorTerms:
        - matchExpressions:
            - key: accelerator
              operator: In
              values: ["nvidia"]
```

2. Run Critical Pods on SSD

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnored...
      nodeSelectorTerms:
        - matchExpressions:
            - key: disktype
              operator: In
              values: ["ssd"]
```

3. Prefer Same Zone (HA)

```
affinity:
  nodeAffinity:
    preferredDuringSchedulingIgnored...
      - weight: 80
        preference:
          matchExpressions:
            - key: topology.kubernetes.io/zone
              operator: In
              values: ["us-east-1a"]
```

4. Isolate by Environment

```
nodeSelector:
  environment: production
tolerations:
  - key: "dedicated"
    operator: "Equal"
    value: "production"
    effect: "NoSchedule"
```

BEST PRACTICES

- ✓ Label your Nodes consistently.
- ✓ Use nodeSelector for simple cases.
- ✓ Use Required Affinity for strict rules.
- ✓ Use Preferred Affinity for flexibility.
- ✓ Combine with Taints & Tolerations for isolation.

COMMON LABELS TO KNOW

- topology.kubernetes.io/zone
- topology.kubernetes.io/region
- node.kubernetes.io/instance-type
- kubernetes.io/os
- kubernetes.io/arch
- disktype, environment, team, workload

REMEMBER

The Scheduler finds the best place for your Pods based on rules, resources, and preferences.

Right rules = Right Pods in Right Nodes!

"Good scheduling is the foundation of a stable and efficient Kubernetes cluster."

VERIQTA @ veriqta

3. TAINTS & TOLERATIONS

"Taints repel Pods. Tolerations allow them."

① TAINTS

- Taints are applied to Nodes.
- They repel Pods that do not have matching Tolerations.
- Used to reserve Nodes for specific workloads.
- A Node can have multiple taints.

TAINT FORMAT

key=value:effect

key : taint key
value : taint value
effect : taint effect
(NoSchedule | PreferNoSchedule | NoExecute)

② TOLERATIONS

- Tolerations are applied to Pods.
- They allow Pods to be scheduled on tainted Nodes.
- A toleration must match the taint key, value and effect.

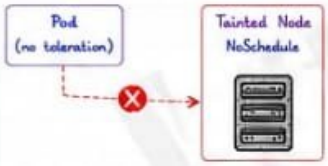
TOLERATION FORMAT (YAML)

```yaml
tolerations:
- key: "key"
  operator: "Equal"   # or Exists
  value: "value"      # if Equal
  effect: "NoSchedule" # required
```

③ TAINT EFFECTS EXPLAINED

1. NoSchedule

- New Pods without toleration will NOT be scheduled on the tainted Node.



Result: Pod will not be scheduled.

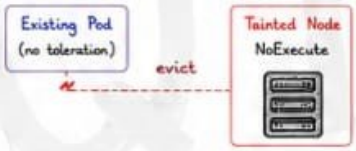
2. PreferNoSchedule

- Kubernetes will try to avoid scheduling Pods without toleration on the tainted Node.
- But it can still schedule if no other Nodes are available.

Result: Soft preference (best-effort).

3. NoExecute

- Existing Pods without toleration will be evicted from the Node.
- New Pods will not be scheduled.



Result: Eviction happens.

IMPORTANT

- For NoExecute, use tolerationSeconds to stay on the Node for a period of time.
- If not set, Pod is evicted immediately.

```yaml
tolerationSeconds: 3600
# 1 hour
```

④ EXAMPLE: TAINTS ON NODES

Node: gpu-node-1

Taints:
- key: nvidia.com/gpu
  value: "present"
  effect: NoSchedule

Node: spot-node-1

Taints:
- key: spot
  value: "true"
  effect: PreferNoSchedule

Node: maintenance-node

Taints:
- key: node.kubernetes.io/maintenance
  value: "true"
  effect: NoExecute

This Pod can run on all three tainted Nodes!

⑤ EXAMPLE: TOLERATIONS IN POD

```yaml
tolerations:
- key: "nvidia.com/gpu"
  operator: "Equal"
  value: "present"
  effect: "NoSchedule"
- key: "spot"
  operator: "Equal"
  value: "true"
  effect: "PreferNoSchedule"
- key: "node.kubernetes.io/maintenance"
  operator: "Equal"
  value: "true"
  effect: "NoExecute"
  tolerationSeconds: 3600
```

⑥ PRODUCTION USE CASES

1. Dedicated Nodes (GPU / ML)

- Taint GPU Nodes with NoSchedule.
- Only ML/AI Pods with toleration can run.
- Prevents other workloads from using expensive GPUs.

Taint: nvidia.com/gpu=present:NoSchedule

2. Spot / Preemptible Nodes

- Taint spot Nodes with PreferNoSchedule.
- Normal Pods avoid them.
- Cost-aware workloads with toleration can still use them.

Taint: spot=true:PreferNoSchedule

3. Maintenance & Drain

- Add NoExecute taint before node drain.
- Existing Pods evicted after tolerationSeconds.
- Ensures graceful maintenance.

Taint: node.kubernetes.io/maintenance=true:NoExecute

4. Critical System Add-ons

- Control plane / system Pods tolerate master taints.
- Ensures they always run where required.
- Example: node-role.kubernetes.io/control-plane

Taint: node-role.kubernetes.io/control-plane=:NoSchedule

⑦ QUICK COMMANDS

```
# Add taint to a node
kubectl taint nodes <node> key=value:NoSchedule
# Remove taint from a node
kubectl taint nodes <node> key:NoSchedule-
# View taints on nodes
kubectl describe node <node> | grep -i taint
# View Pod tolerations
kubectl describe pod <pod> | grep -i tolerations
```

⑧ KEY TAKEAWAYS

- ✓ Taints are set on Nodes to repel Pods.
- ✓ Tolerations are set on Pods to allow them.
- ✓ NoSchedule = hard block for new Pods.
- ✓ PreferNoSchedule = soft preference.
- ✓ NoExecute = evict existing Pods.
- ✓ Use taints to reserve Nodes for the right workloads.

"Right workload. Right Node. Every time."

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES PRODUCTION ENGINEERING

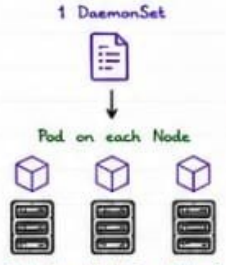
4. DAEMONSETS

"Ensure that a copy of a Pod runs on ALL (or selected) Nodes."

1 WHAT IS A DAEMONSET?

- A DaemonSet ensures that all (or some) Nodes run a copy of a Pod.
- As Nodes join the cluster, Pods are automatically created.
- As Nodes leave, Pods are automatically removed.
- Used for cluster-wide background services (node-level agents).

HOW IT WORKS



KEY POINT

Unlike Deployments, DaemonSets are NOT for user-facing applications.

2 COMMON NODE AGENTS

AGENT TYPE	PURPOSE
Logging Agents (e.g., Fluentd, Vector)	• Collect logs from /var/log and application logs. • Ship to centralized logging system (ELK, Loki, CloudWatch, etc.)
Monitoring Agents (e.g., Node Exporter, Telegraf)	• Collect metrics like CPU, Memory, Disk, Network. • Expose metrics for Prometheus, Grafana, etc.
Security Agents (e.g., Falco, Sysdig)	• Runtime security & threat detection. • Monitor syscalls, file access, etc.
Network Agents (e.g., CNI plugins, Calico Node)	• Provide networking on each Node. • Manage routing, policies, connectivity.
Storage / CSI Node Plugins	• Enable volumes on each Node. • CSI node plugins, device plugins.

3 DAEMONSET LIFECYCLE

1. **Create** — You create a DaemonSet manifest.
2. **Schedule** — Kubernetes Scheduler places a Pod on each eligible Node.
3. **Run** — Pod runs on the Node as a long-running agent.
4. **Node Joins** — New Node joins cluster → Pod is created automatically.
5. **Node Leaves** — Node leaves cluster → Pod is removed automatically.
6. **Update** — DaemonSet updates Pods rolling on each Node.

UPDATE STRATEGY

RollingUpdate (default) → Updates Pods one by one per Node.
OnDelete → You manually delete Pods for updates.

POD ISOLATION

DaemonSet Pods run by default in the host's network, PID namespace and can access hostPath if needed.

4 DAEMONSET EXAMPLE (YAML)

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      containers:
      - name: node-exporter
        image: prom/node-exporter:latest
        ports:
          - containerPort: 9100
            hostPort: 9100
        args:
        - --path.rootfs=/host
        volumeMounts:
          - name: host
            mountPath: /host
            readOnly: true
      volumes:
        - name: host
          hostPath:
            path: /
            type: Directory
```

EXPLANATION

- ✓ Runs node-exporter on every Node.
- ✓ Uses hostPath mount to access host root.
- ✓ Exposes metrics on port 9100.
- ✓ Collects real Node metrics for Prometheus.

5 PRODUCTION EXAMPLES

Logging DaemonSet	Fluentd/Vector runs on all Nodes, collects logs and ships to central logging (e.g., Elasticsearch, Loki, CloudWatch).
Monitoring DaemonSet	Node Exporter / Telegraf runs on all Nodes, exposes metrics for Prometheus & Grafana dashboards.
Security DaemonSet	Falco/Sysdig detects suspicious activity on each Node in real-time and alerts.
Storage Plugin DaemonSet	CSI Node plugin runs on each Node to manage volumes & storage devices.
Custom Node Agents	Custom scripts, backup agents, license managers, or compliance scanners on every Node.

6 BEST PRACTICES

- ✓ Use DaemonSets only for node-level agents.
- ✓ Keep Pods lightweight and resource-efficient.
- ✓ Use tolerations to run on tainted Nodes when required.
- ✓ Monitor DaemonSet Pods – they are critical for cluster health.
- ✓ Use hostPath carefully and read-only when possible.

COMMON LABELS

- app
- component: agent
- role: node
- env: production
- team: platform

REMEMBER

- ✓ DaemonSet = one Pod per Node.
- ✓ Great for agents, not for user apps.
- ✓ Automatic lifecycle with Nodes.
- ✓ Essential for observability, security and operations.

"Great clusters run great agents on every Node."

VERIQTA @ veriqta

VOLUME 2 KUBERNETES PRODUCTION ENGINEERING

5. STATEFULSETS

"Stable identity. Stable storage. Stable systems."

1 WHAT IS A STATEFULSET?

- A StatefulSet manages stateful applications in Kubernetes.
- Provides stable network identities, persistent storage, and ordered deployment & scaling.
- Ideal for databases, message brokers, and other stateful workloads.

STATEFULSET vs DEPLOYMENT

Deployment	StatefulSet
• Stateless	• Stateful
• Random pod names	• Stable pod names
• No identity guarantees	• Stable identities
• Not ordered	• Ordered
• No stable storage	• Stable storage

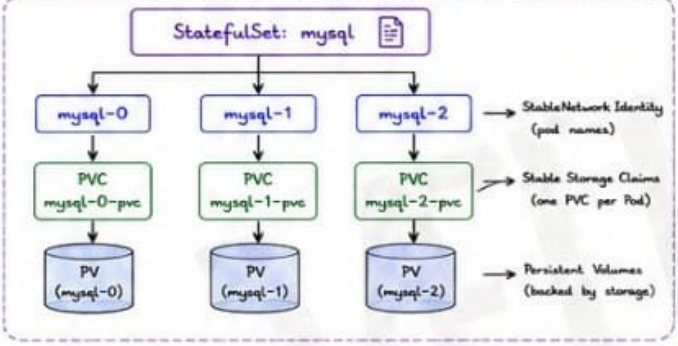
KEY POINT

StatefulSet gives each Pod a sticky identity and its own storage.

2 CORE FEATURES

Stable Identities	Stable hostname & network identity: <statefulset-name>-<ordinal> (e.g., mysql-0, mysql-1, mysql-2)
Persistent Storage	Each Pod gets its own PersistentVolume Claim (PVC) that is preserved across restarts and rescheduling.
Ordered Deployment	Pods are created in order (0 → 1 → 2...) and each waits for the previous to be Ready.
Ordered Scaling & Deletion	Scaling up/down and deletion happen in reverse order to maintain consistency.
Predictable & Reliable	Built for stateful workloads that require consistency, durability and ordering.

3 STATEFULSET ARCHITECTURE



4 ORDERED DEPLOYMENT & SCALING

DEPLOYMENT ORDER (Scaling Up)

1 Create Pod-0 → 2 Wait for Ready → 3 Create Pod-1 → 4 Wait for Ready → 5 Create Pod-2 → ...

SCALINGG DOWN ORDER (Reverse Order)

1 Delete Pod-2 → 2 Wait for Terminated → 3 Delete Pod-1 → 4 Wait for Terminated → 5 Delete Pod-0

Ensures data consistency and avoids split-brain issues.

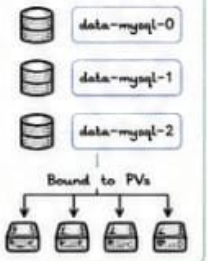
5 PERSISTENT STORAGE PER POD

- Each Pod gets a dedicated PVC.
- PVC name format: `<claimTemplateName>-<statefulsetName>-<ordinal>`
- PVCs are NEVER shared between Pods.
- When a Pod is deleted, its PVC (and PV) is retained by default.

VOLUME CLAIM TEMPLATE (YAML)

```yaml
volumeClaimTemplates:
- metadata:
    name: data
  spec:
    accessModes: ["ReadWriteOnce"]
    storageClassName: gp3
    resources:
      requests:
        storage: 20Gi
```

EXAMPLE PVCs



6 STABLE NETWORK IDENTITIES

- Each Pod gets a DNS record: `<pod-name>.<headless-service>.<namespace>.svc.cluster.local`
- Example:
  - Pod name: mysql-1
  - Headless Service: mysql
  - Namespace: prod
  - DNS: mysql-1.mysql.prod.svc.cluster.local
- Used for internal communication between database nodes.

HEADLESS SERVICE (YAML)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: mysql
  namespace: prod
spec:
  clusterIP: None
  selector:
    app: mysql
  ports:
  - port: 3306
```

7 PRODUCTION DATABASE EXAMPLES

MySQL
- Primary / Replica
- Galera Cluster
- Stable identities
- Persistent data

StatefulSet + Headless Service + PVC

PostgreSQL
- Primary / Standby
- Patroni / Stolon
- Ordered deployment
- Data durability

StatefulSet + PVC + Readiness Probes

MongoDB
- Replica Set
- Stable members
- Persistent volumes
- Ordered scaling

StatefulSet + PVC + Headless Service

Kafka
- Brokers
- Persistent logs
- Stable IDs
- Ordered rollout

StatefulSet + PVC + Node Affinity

Elasticsearch
- Master / Data Nodes
- Persistent data
- Cluster stability
- Ordered updates

StatefulSet + PVC + Anti-Affinity

8 BEST PRACTICES

- ✓ Use StatefulSets for stateful workloads only.
- ✓ Always use headless Service for stable DNS.
- ✓ Use volumeClaimTemplates for persistent storage.
- ✓ Configure liveness & readiness probes.
- ✓ Use PodDisruptionBudgets for HA workloads.
- ✓ Test failover & recovery regularly.

9 COMMON USE CASES

- ✓ Databases (MySQL, PostgreSQL, MongoDB)
- ✓ Message Brokers (Kafka, RabbitMQ)
- ✓ Search Engines (Elasticsearch, OpenSearch)
- ✓ Distributed Systems (Consul, ZooKeeper)
- ✓ Time-series DBs (Cassandra, InfluxDB)

10 REMEMBER

- ✓ Stable identity = predictable communication.
- ✓ Persistent storage = durable data.
- ✓ Ordered operations = consistency.
- ✓ StatefulSets are the backbone of production-grade stateful systems.

"StatefulSets bring order, identity, and durability to stateful applications."

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES
PRODUCTION
ENGINEERING

6. JOBS & CRONJOBS

"Run it once. Run it later. Run it on schedule."

1 WHAT ARE JOBS?

- Jobs create Pods to run a task and ensure it completes successfully.
- Designed for finite tasks (batch, data, cleanup, backups).
- The Job succeeds when the required Pods complete.
- After completion, Pods are cleaned up (based on policy).

WHEN TO USE JOBS?

- Data migration
- Database backups
- Report generation
- File processing
- ML training
- One-time tasks

2 JOB LIFECYCLE

Create Job → Controller creates Pods → Pods Run Task → Job Completes → Pods Cleaned Up

JOB CONDITIONS

- **Active** → Job is running (at least one Pod active).
- **Succeeded** → All Pods completed successfully.
- **Failed** → Job failed (exceeded retries or Pod failed).

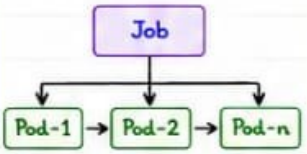
KEY POINT

Jobs are for finite tasks, not for services or long-running workloads.

3 TYPES OF JOBS

SINGLE JOB (Default)

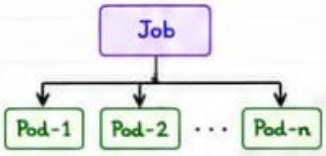
Runs Pods one after another until completion.



completions: 1 (default)
parallelism: 1 (default)

PARALLEL JOBS

Runs multiple Pods at the same time to speed up.



completions: n (total successful Pods needed)
parallelism: m (Pods run in parallel)

4 CRONJOBS (SCHEDULED JOBS)

- ✓ CronJobs create Jobs on a schedule using cron expressions.
- ✓ Perfect for recurring tasks.
- ✓ Each schedule triggers a new Job.

CRON EXPRESSION FORMAT

```
*  *  *  *  *
Min (0-59)
Hour (0-23)
Day (1-31)
Month (1-12)
Day of Week (0-6) (Sun=0)
```

EXAMPLES

- `0 * * * *` → Every hour
- `0 2 * * *` → Every day at 2 AM
- `0 0 * * 0` → Every Sunday at midnight
- `30 1 * * 1-5` → Weekdays at 1:30 AM

5 EXAMPLE: JOB vs CRONJOB (YAML)

JOB EXAMPLE

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: backup-job
spec:
  completions: 1        # Total successful Pods needed
  parallelism: 2        # How many Pods run in parallel
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: backup
        image: busybox
        command: ["sh", "-c", "tar -czf /backup/data.tgz /data"]
```

CRONJOB EXAMPLE

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: daily-report
spec:
  schedule: "0 2 * * *"  # Every day at 2 AM
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
          - name: report
            image: alpine
            command: ["sh", "-c", "python generate_report.py"]
```

6 JOB SPEC (KEY FIELDS)

FIELD	DESCRIPTION
completions	Number of successful Pods needed for the Job.
parallelism	Number of Pods that can run in parallel.
backoffLimit	Maximum number of retries for a failed Pod.
activeDeadlineSeconds	Max time (in seconds) for the Job to run.
ttlSecondsAfterFinished	Automatically clean up Job & Pods after completion.
restartPolicy	Should be Never or OnFailure (usually Never).

7 PRODUCTION EXAMPLES

Database Backup

- Dump DB to S3 or storage.
- Run daily using CronJob.

Log Cleanup

- Delete old logs from server.
- Keep disk clean & optimized.

Report Generation

- Generate business reports.
- Email or upload to dashboard.

Data Processing

- Process large files or datasets.
- Run in parallel for speed.

ML / AI Training

- Train models on GPU nodes.
- One-time or scheduled runs.

8 JOBS vs CRONJOBS

	JOBS	CRONJOBS
Purpose	Run a task once	Run tasks on a schedule
Trigger	Manual / API / CI Pipeline	Cron Schedule (time-based)
Creates	Pods	Jobs (which create Pods)
Repeat	No (one-time)	Yes (recurring)
Best for	Batch jobs, one-time tasks	Recurring jobs, automation

9 BEST PRACTICES

- ✓ Use Jobs for finite tasks, not long-running services.
- ✓ Set backoffLimit to avoid endless retries.
- ✓ Use ttlSecondsAfterFinished to auto-clean resources.
- ✓ Keep Jobs idempotent (safe to run multiple times).
- ✓ Monitor Job status with kubectl or Prometheus.

COMMON LABELS

- app: backup-job
- component: batch
- env: production
- team: platform
- type: job / cronjob

11 REMEMBER

- ✓ Jobs = Run it once (to completion).
- ✓ CronJobs = Run it on a schedule.
- ✓ Parallelism speeds up processing.
- ✓ Consistency + Cleanup = Production Ready.

"Automate the boring. Schedule the important. Let Kubernetes handle the rest."

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES
PRODUCTION
ENGINEERING

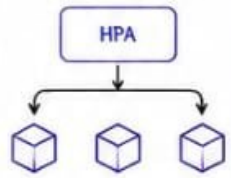
7. HORIZONTAL POD AUTOSCALER (HPA)

"Scale smart. Scale automatically. Scale with HPA."

1 WHAT IS HPA?

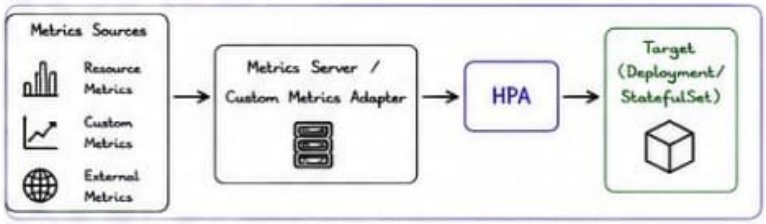
- Horizontal Pod Autoscaler (HPA) automatically adjusts the number of Pods in a Deployment, StatefulSet or ReplicaSet based on resource usage or custom metrics.
- Ensures applications stay responsive under varying load.
- Saves cost by scaling down when demand is low.

HPA AT A GLANCE



Automatically adjusts replica count

2 HPA ARCHITECTURE



COMPONENTS

- Metrics Server – collects CPU & Memory metrics from Kubelet.
- HPA Controller – makes scaling decisions.
- Custom Metrics Adapter – exposes application/external metrics.
- Target – the workload to be scaled.

3 METRICS SERVER

- Metrics Server collects resource metrics from Kubelet on each Node.
- It stores metrics in memory and makes them available to the HPA.
- Required for CPU and Memory based autoscaling.

INSTALL METRICS SERVER (YAML)

```
# Install Metrics Server using Helm
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm repo update
helm upgrade --install metrics-server metrics-server/metrics-server \
 --namespace kube-system --create-namespace
# Verify
kubectl top nodes
kubectl top pods -A
```

4 CPU SCALING (EXAMPLE)

HPA BASED ON CPU UTILIZATION

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: webapp
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60   # target average CPU %
```

HOW IT WORKS

- If average CPU > 60% → scale up
- If average CPU < 60% → scale down
- HPA continuously monitors & adjusts replicas.

5 MEMORY SCALING (EXAMPLE)

HPA BASED ON MEMORY UTILIZATION

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webapp-memory-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: webapp
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 70   # target average Memory %
```

NOTES

- Memory metrics are based on container memory usage.
- Ensure proper resource requests & limits.

Memory Utilization Target: 70%

6 CUSTOM METRICS (EXAMPLE)

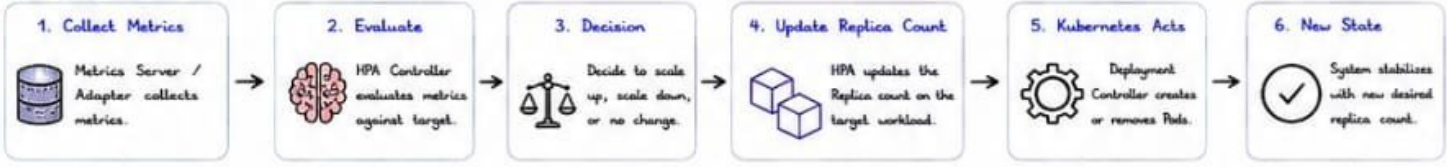
HPA BASED ON CUSTOM METRIC (REQUESTS PER SECOND)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webapp-rps-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: webapp
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Pods
    pods:
      metric:
        name: http_requests_per_second
      target:
        type: AverageValue
        averageValue: "100"   # target 100 RPS per Pod
```

CUSTOM METRICS SOURCES

- Prometheus Adapter
- Datadog Adapter
- CloudWatch Adapter
- Any external system exposing metrics

7 SCALING WORKFLOW



8 BEST PRACTICES

- ✓ Set proper resource requests & limits.
- ✓ Choose realistic minReplicas & maxReplicas.
- ✓ Avoid aggressive scaling (set stabilization window).
- ✓ Use custom metrics for application specific needs.
- ✓ Monitor HPA events & metrics.

COMMON LABELS

- app: webapp
- component: api
- tier: backend
- env: production

9 PRODUCTION EXAMPLES

- ✓ E-commerce site – scale on CPU & RPS
- ✓ API Server – scale on custom request metrics
- ✓ Batch Processing Service – scale on queue length
- ✓ Microservices – autoscale individual services

"Let HPA handle the spikes. You focus on building greatness."

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES PRODUCTION ENGINEERING

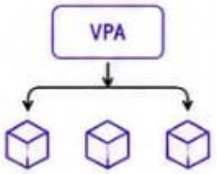
8. VERTICAL POD AUTOSCALER (VPA)

"Right size today. Run better tomorrow."

1 WHAT IS VPA?

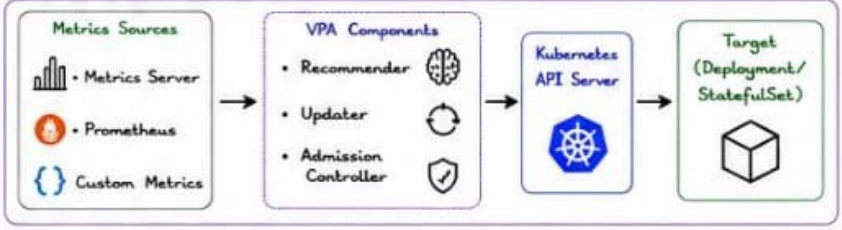
- Vertical Pod Autoscaler (VPA) automatically adjusts the resource requests (CPU/Memory) of containers.
- It recommends or updates the resources based on actual usage history.
- VPA does NOT change the number of Pods, it changes the size of Pods.

VPA AT A GLANCE



Automatically adjusts resource requests (CPU & Memory)

2 VPA ARCHITECTURE



COMPONENTS

- **Metrics Server** – collects CPU & Memory metrics from Kubelet.
- **Recommender** – analyzes usage history & generates recommendations.
- **Updater** – updates resource requests (evicts Pods if needed).
- **Admission Controller** – prevents Pods with lower resources than recommended.

3 RESOURCE RECOMMENDATIONS

- VPA analyzes historical resource usage and recommends optimal CPU & Memory requests.
- Recommendations are generated per container.
- Helps prevent under-provisioning (OOMKills) and over-provisioning (wasted resources).

EXAMPLE RECOMMENDATION (kubectl describe vpa)

Container	CPU (cores)		Memory (Mi)	
	Current	Recommended	Current	Recommended
webapp	100m	550m	256Mi	512Mi
Status: Recommendation provided ✓				

4 AUTOMATIC UPDATES

- In Auto mode, VPA automatically updates resource requests.
- It evicts Pods one by one to apply new requests.
- New Pods are created with the updated resource requests.
- No manual changes required.

AUTO UPDATE WORKFLOW

5 UPDATE MODES

MODE	DESCRIPTION	BEHAVIOR	USE CASE
Off (Recommendations only)	VPA only provides recommendations.	No automatic updates.	Observe & analyze before applying changes.
Initial (One-time update)	Updates resource requests only when a Pod is created.	No updates after initial creation.	Good for batch jobs or stable workloads.
Auto (Continuous update)	Continuously updates resource requests.	Evicts Pods to apply new requests.	For dynamic workloads that need ongoing optimization.

6 BEST PRACTICES

- ✓ Start with Off mode to observe recommendations.
- ✓ Ensure Metrics Server or Prometheus is healthy.
- ✓ Set appropriate resource requests & limits. (VPA works best when both are defined.)
- ✓ Use PodDisruptionBudget (PDB) to maintain availability during evictions.
- ✓ Monitor VPA events & recommendations regularly.
- ✓ Do not use VPA with HPA for the same container (they serve different purposes).
- ✓ Test in staging before enabling Auto mode in production.

Right size resources, right cost, right performance.

7 PRODUCTION CONSIDERATIONS

Evictions Impact	Auto mode evicts Pods. Use PDBs and plan for sufficient replicas to avoid downtime.
Rolling Updates	VPA + Deployments = rolling evictions. Plan for gradual resource adjustments.
Resource Limits	Set sensible limits. VPA respects limits and won't exceed them.
Monitoring	Monitor VPA status, events, and recommendations using kubectl or dashboards.
Start Small	Begin with non-critical workloads. Scale confidence, then go Auto mode.

8 VPA VS HPA

FEATURE	VPA	HPA
Purpose	Adjusts resource requests	Adjusts number of Pods
Metric	CPU, Memory, Custom	CPU, Memory, Custom
Action	Evicts Pods, updates requests	Adds/Removes Pods
Works with	Inside a Pod	Across multiple Pods
Use together?	⊗ Not for the same container (may cause conflicts)	✓ Use together with different metrics/apps

9 PRODUCTION EXAMPLES

Web Applications
Automatically right-size frontend & backend workloads based on traffic patterns.

Microservices
Optimize each service based on real usage. Reduce waste & improve efficiency.

Batch Jobs
Set Initial mode to get correct resources for jobs.

Data Processing
Handle varying data volumes by adjusting resource requests automatically.

Cost Optimization
Right-size resources to reduce cloud costs without impacting performance.

"VPA recommends the best size. You let it apply the magic."

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES PRODUCTION ENGINEERING

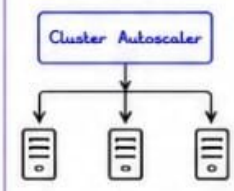
9. CLUSTER AUTOSCALER

"Right size your nodes. Optimize cost. Keep performance."

1 WHAT IS CLUSTER AUTOSCALER?

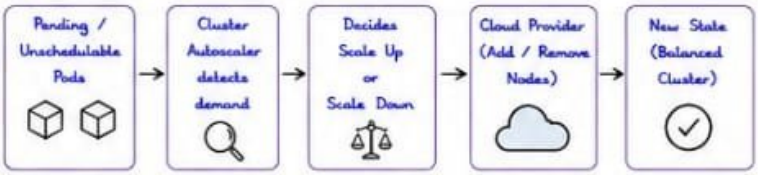
- Cluster Autoscaler (CA) automatically adjusts the number of nodes in a Kubernetes cluster.
- It watches for unschedulable Pods and modifies the node group size.
- Works with Cloud Provider API to add or remove nodes.
- It does NOT change the number of Pods, only nodes.

AT A GLANCE



Automatically adjusts node count

2 HOW IT WORKS (OVERVIEW)



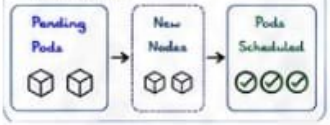
REQUIREMENTS

- Nodes must be part of an Auto Scaling Group / Node Group.
- Nodes must have proper labels & taints (if needed).
- Scheduler must be able to place Pods on new nodes.

3 NODE SCALING

SCALE UP (Add Nodes)

- Trigger: Pending Pods cannot be scheduled due to insufficient resources.
- CA checks node groups and finds the best fit.
- Requests cloud provider to provision new nodes.
- Scheduler places Pods on new nodes.



SCALE DOWN (Remove Nodes)

- Trigger: Underutilized nodes (low CPU/Memory usage).
- CA marks nodes as candidates for removal.
- Drains nodes (evict Pods safely).
- Requests cloud provider to terminate nodes.

4 CLOUD INTEGRATION

Cluster Autoscaler integrates with Cloud Provider APIs.

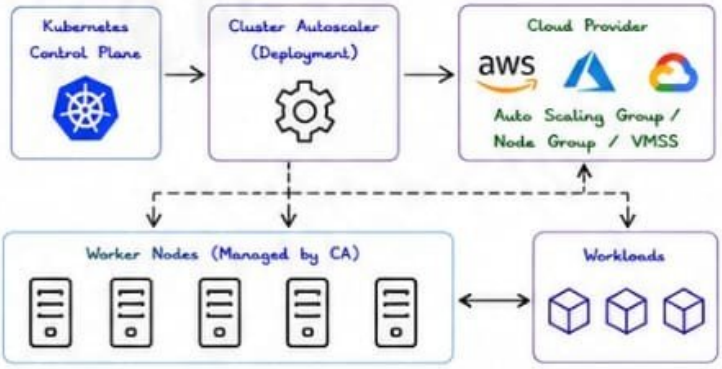
AWS	Uses Auto Scaling Groups (ASG) & Launch Templates. Adds/Removes EC2 instances.
GCP	Uses Managed Instance Groups (MIG). Adds/Removes Compute Engine instances.
Azure	Uses Virtual Machine Scale Sets (VMSS). Adds/Removes VM instances.
Other	Supports custom providers via cloudprovider interface.

5 COST OPTIMIZATION

- Scale up only when needed → handle spikes.
- Scale down idle nodes → reduce cloud costs.
- Supports spot/preemptible instances.
- Works well with bin-packing & VPA.
- Helps maintain right-sized infrastructure.

Pay for what you use. Nothing more.

6 PRODUCTION ARCHITECTURE



7 CLUSTER AUTOSCALER COMPONENTS

COMPONENT	DESCRIPTION
CA Pod	Runs as a Deployment in the cluster.
Cloud Provider	Provides API to manage nodes.
Node Groups	Groups of nodes that can be scaled.
Scheduler	Places Pods on available nodes.
Metrics	CA uses resource requests, not actual usage.

8 KEY CONFIGURATION (FLAGS EXAMPLES)

```
--cloud-provider=aws
--nodes=1:10:asg:us-east-1:123456789012:autoScalingGroupName/my-asg
--scale-down-enabled=true
--scale-down-delay-after-add=10m
--scale-down-unneeded-time=10m
--expander=least-waste      # options: least-waste, most-pods, random, price
--balance-similar-node-groups=true
```

9 BEST PRACTICES

- ✓ Set proper resource requests & limits on Pods.
- ✓ Use multiple node groups (on-demand + spot).
- ✓ Enable scale down with safe delays.
- ✓ Use PodDisruptionBudgets (PDB) to protect critical apps.
- ✓ Monitor CA logs, events & metrics.
- ✓ Test scale up & scale down in non-prod first.

10 PRODUCTION EXAMPLES

"Let the cluster autoscaler handle the nodes, so you can focus on your application."

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES PRODUCTION ENGINEERING

10. KUBERNETES PROBES

"Know your app's health. Keep your users happy."

1 WHAT ARE PROBES?

- Probes are used by Kubernetes to check the health of your containers.
- They help Kubernetes determine when to restart, route traffic or start containers.
- Three types: Liveness, Readiness, Startup.

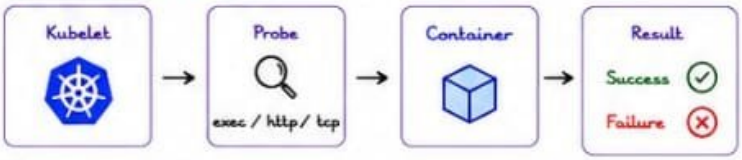
PROBES AT A GLANCE

Liveness Probe
Is the container alive?

Readiness Probe
Is the container ready to serve traffic?

Startup Probe
Is the application started?

2 HOW PROBES WORK



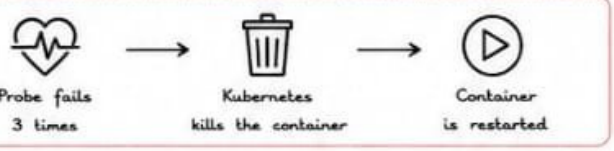
Probe types:
- Exec – runs a command inside the container.
- HTTP – sends an HTTP request to an endpoint.
- TCP – opens a TCP connection to a port.

3 LIVENESS PROBE

- Checks if the container is running.
- If it fails, Kubernetes kills the container and restarts it.
- Use for deadlock detection, app hangs, memory leaks, etc.

YAML EXAMPLE (HTTP)

```yaml
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
```



4 READINESS PROBE

- Checks if the container is ready to serve traffic.
- If it fails, the Pod is removed from Service endpoints.
- Use during startup, dependency checks, warm-up, etc.

YAML EXAMPLE (HTTP)

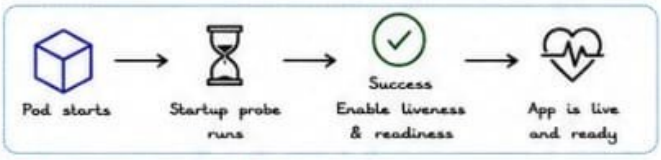
```yaml
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
  timeoutSeconds: 1
  failureThreshold: 3
```

5 STARTUP PROBE

- Used to check if the application has started.
- Disables liveness and readiness until startup probe succeeds.
- Great for slow-starting applications.

YAML EXAMPLE (HTTP)

```yaml
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  initialDelaySeconds: 0
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 30
```



6 PROBE FAILURES – WHAT HAPPENS?

PROBE TYPE	FAILURE EFFECT	KUBERNETES ACTION	TRAFFIC IMPACT
Liveness	Container is unhealthy	Kills & restarts	Pod restarts, traffic lost briefly
Readiness	Container not ready	Removes from Service endpoints	No traffic sent to Pod
Startup	App not started yet	Keeps checking until success	No traffic until startup succeeds

EXAMPLE SCENARIO
If your app takes 2 minutes to start and you don't use Startup Probe, Liveness Probe may fail and restart the container repeatedly.

7 PROBE TIMING PARAMETERS

PARAMETER	DESCRIPTION	TIP
initialDelaySeconds	Time to wait before first probe	Give app time to start
periodSeconds	How often to run the probe	Balanced frequency
timeoutSeconds	Timeout for each probe	Keep short (1–3s)
failureThreshold	Failures before action	Don't set too low
successThreshold	Successes before ready	Usually 1 (default)

8 BEST PRACTICES

- ✓ Always use Readiness Probe for all apps.
- ✓ Use Startup Probe for slow-starting applications.
- ✓ Avoid heavy exec commands – keep probes lightweight.
- ✓ Use meaningful endpoints: /healthz, /ready, /startup.
- ✓ Set realistic timings (don't probe too aggressively).
- ✓ Monitor probe failures in production.

Good probes = Happy users + Stable production

9 PRODUCTION TIPS

- Design app-specific endpoints for health, ready and startup.
- Monitor probe metrics in Prometheus / Grafana.
- Alert on high probe failure rates.
- Test probes before deploying to production.
- Don't expose probe endpoints publicly (use Internal Services).

"Probes protect your application, and your application protects your business."

VERIQTA @ veriqta

VOLUME 2 KUBERNETES PRODUCTION ENGINEERING

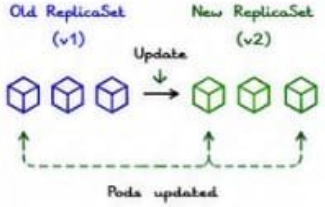
11. ROLLING UPDATES & ROLLBACKS

"Deploy with confidence. Roll back with ease."

1 WHAT IS A ROLLING UPDATE?

- Kubernetes updates Pods gradually instead of all at once.
- Ensures zero downtime and high availability.
- Old Pods are replaced by new Pods step by step.

ROLLING UPDATE FLOW



2 ROLLING UPDATE STRATEGY

- Kubernetes Deployment updates Pods using strategy: RollingUpdate (default)

KEY POINTS

- New Pods are started before old Pods are terminated.
- Traffic is always routed to Ready Pods only.
- Ensures application stays available during update.

3 maxUnavailable

- Controls how many Pods can be unavailable during the update.
- Can be an absolute number or a percentage.
- Default: 25%

EXAMPLES

REPLICAS	maxUnavailable	MAX PODS UNAVAILABLE
4	25% (default)	1
4	1	1
4	2	2
10	30%	3

Tip
Higher value = faster update but higher risk.

4 maxSurge

- Controls how many extra Pods can be created above the desired replica count.
- Can be an absolute number or a percentage.
- Default: 25%

EXAMPLES

REPLICAS	maxSurge	MAX EXTRA PODS
4	25% (default)	1
4	1	1
4	2	2
10	30%	3

Tip
Higher value = faster update but uses more resources.

5 ROLLING UPDATE BEHAVIOR (EXAMPLE)

Deployment: 4 replicas, maxUnavailable: 25% (1), maxSurge: 25% (1)

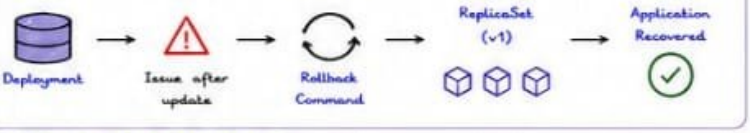
STEP	ACTION	OLD PODS (v1)	NEW PODS (v2)	TOTAL PODS
1	Start update	4	0	4
2	Create new pod	3	1	5
3	Terminate old pod	3	1	4
4	Create new pod	2	2	4
5	Terminate old pod	2	2	4
6	Create new pod	1	3	4
7	Terminate old pod	1	3	4
8	Create new pod	0	4	4

At no point are more than 5 Pods or less than 3 Pods running.

6 ROLLBACKS

- If something goes wrong, rollback to the previous ReplicaSet.
- Kubernetes restores the previous version automatically.

ROLLBACK FLOW



COMMON ROLLBACK COMMANDS

```
# Rollback to previous revision
kubectl rollout undo deployment/<name>

# Rollback to specific revision
kubectl rollout undo deployment/<name> --to-revision=2

# Check rollout status
kubectl rollout status deployment/<name>
```

7 REVISION HISTORY

- Kubernetes keeps history of ReplicaSets.
- Useful for rollbacks & debugging.
- Default history limit: 10

```
kubectl rollout history deployment/<name>

kubectl rollout history deployment/<name> --revision=2
```

REVISION EXAMPLE

REVISION	CHANGE-CAUSE	IMAGE
1	Initial Version	v1.0
2	Update login API	v1.1
3	Fix memory leak	v1.2

8 DEPLOYMENT SAFETY

- ✓ Set proper resource requests & limits.
- ✓ Use liveness, readiness & startup probes.
- ✓ Test in staging before production.
- ✓ Use small maxSurge & maxUnavailable for critical apps.
- ✓ Monitor rollout using dashboards & alerts.
- ✓ Keep revision history for quick rollback.

Safe deploys + quick rollbacks = happy users ☺

9 PRODUCTION BEST PRACTICES

- Use RollingUpdate strategy for zero downtime.
- Tune maxUnavailable & maxSurge carefully.
- Monitor metrics & logs during rollout.
- Run automated tests before promotion.
- Have rollback plan ready.
- Keep your app stateless when possible.

"Great deployments are not about speed, they are about safety."

VERIQTA @ veriqta

VOLUME 2 KUBERNETES PRODUCTION ENGINEERING

12. BLUE-GREEN & CANARY DEPLOYMENTS

"Deliver new features. Reduce risk. Keep users happy."

1 WHAT ARE THEY?

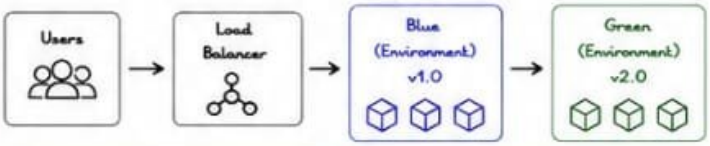
- Advanced deployment strategies that minimize risk and downtime.
- Allows safe release of new versions to production.
- Easier rollback and better control over traffic.
- Used with Services, Ingress, Service Mesh (Istio), or Load Balancers.

STRATEGY COMPARISON

STRATEGY	RISK	TRAFFIC CONTROL	ROLLBACK	USE CASE
Rolling Update	Medium	Limited	Medium	Simple updates
Blue-Green	Low	Easy (All/None)	Instant	Major releases
Canary	Very Low	Granular (%)	Easy	New features
Progressive	Very Low	Incremental	Easy	Large scale rollouts

2 BLUE-GREEN DEPLOYMENT

Two identical environments: Blue (current) and Green (new).



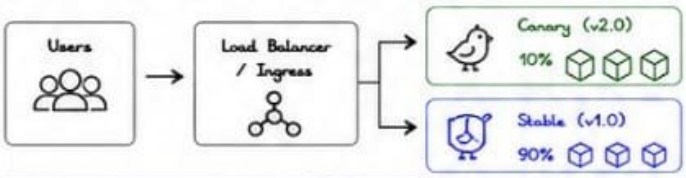
FLOW

KEY POINTS

- Zero downtime (instant traffic switch).
- Full version runs in parallel.
- Rollback is instant (switch back to Blue).
- Requires 2x infrastructure.

3 CANARY DEPLOYMENT

Expose new version to a small subset of users first.



FLOW

KEY POINTS

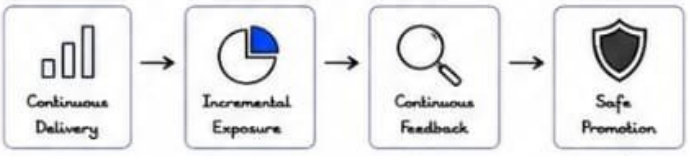
- Start small (1–10% traffic).
- Monitor system metrics, errors, and user behavior.
- Increase traffic gradually.
- Rollback by shifting traffic back to stable version.

TRAFFIC SPLIT EXAMPLE

STAGE	CANARY %
Initial	5%
Step 1	25%
Step 2	50%
Step 3	75%
Full	100%

4 PROGRESSIVE DELIVERY

A broader term that includes Canary, Blue-Green & A/B testing.



TOOLS THAT HELP

Tip

Start small. Measure everything. Automate rollback. Ship with confidence.

5 TRAFFIC SHIFTING METHODS

- Service Selector (Blue-Green switch)
- Ingress Annotations (NGINX/ALB)
- Service Mesh (Istio/Linkerd)
- Feature Flags (LaunchDarkly/Flagsmith)

METHOD	DESCRIPTION
Service Selector	Switch Service selector from Blue label to Green label.
Ingress	Use annotations (nginx.ingress.kubernetes.io/canary) to split traffic.
Service Mesh	Use VirtualService (Istio) to define traffic weights.
Feature Flags	Hide/show features based on user segment or percentage.

6 ROLLBACK STRATEGY

BLUE-GREEN ROLLBACK

- Issue detected on Green
- ↓ Switch traffic back to Blue
- ↓ Investigate & fix Green

CANARY ROLLBACK

- High error rate in Canary
- ↓ Shift 100% traffic back to Stable
- ↓ Fix & try again

AUTOMATE ROLLBACK

- ☑ Use alerts (Prometheus + Alertmanager)
- ☑ Define error rate thresholds (e.g. >5%)
- ☑ Automate rollback with Argo Rollouts / Flagger
- ☑ Notify team (Slack/Email)

7 PRODUCTION EXAMPLES

E-COMMERCE APP
Use Blue-Green for major releases (e.g. v2.0). Rollback instantly if checkout fails.

MOBILE BACKEND
Use Canary to release new API to 5% of users in one region first.

SaaS PLATFORM
Use progressive delivery with Istio. Increase traffic 10% every 10 minutes.

FINANCIAL APP
Blue-Green for compliance releases. Canary for minor features.

8 BEST PRACTICES

- ✓ Always test in staging before production.
- ✓ Monitor SLIs, metrics, logs, and user behavior.
- ✓ Define clear success & failure criteria.
- ✓ Automate rollbacks.
- ✓ Use health checks & probes.
- ✓ Document your rollout & rollback plan.

SUCCESS METRICS

- ☑ High availability
- ☑ Low error rate
- ☑ Fast rollback time
- ☑ Happy users 😊

"Great deployments are shipped gradually, measured carefully, and rolled back instantly." VERIQTA

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES PRODUCTION ENGINEERING

13. RESOURCE OPTIMIZATION

"Right size, every time. Optimize resources. Maximize value."

1 CPU OPTIMIZATION

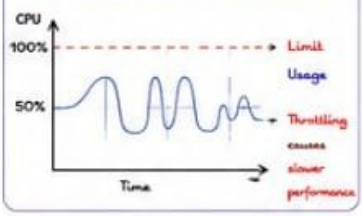
- Request the CPU you need.
- Limit the CPU you can burst.
- Avoid CPU throttling.
- Monitor usage & right size.
- Use HPA for auto scaling.

CPU UNITS

1 CPU = 1000m

250m = 0.25 CPU
500m = 0.5 CPU
1000m = 1 CPU
2000m = 2 CPU

CPU THROTTLING (When limit is hit)



BEST PRACTICES

- ✓ Set realistic requests & limits.
- ✓ Avoid very low CPU limits.
- ✓ Use HPA to scale out.
- ✓ Monitor throttling events.

2 MEMORY OPTIMIZATION

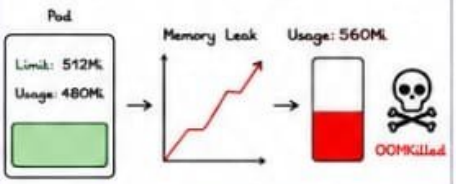
- Memory is not overcommitted in Kubernetes.
- If usage > limit → OOMKilled.
- Right size memory to avoid waste & OOM.
- Monitor memory usage & leaks.

MEMORY UNITS

1 Gi = 1024 Mi

Examples:
256Mi = 0.25Gi
512Mi = 0.5Gi
1Gi = 1024Mi

OOMKILLED EXAMPLE



BEST PRACTICES

- ✓ Set memory requests & limits.
- ✓ Leave headroom for spikes.
- ✓ Use VPA for recommendations.
- ✓ Fix memory leaks early.

3 REQUESTS vs LIMITS

TYPE	REQUESTS	LIMITS	PURPOSE
CPU	Minimum CPU guaranteed	Maximum CPU can use	Scheduling & bursting
MEMORY	Minimum Memory guaranteed	Maximum Memory can use	Scheduling & protection

Requests = what you need to run.

Limits = what you don't want to exceed.

EXAMPLE

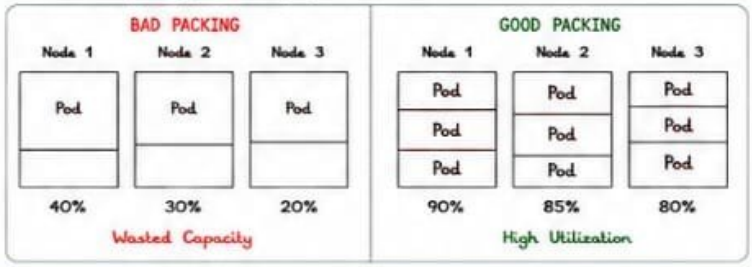
```
resources:
  requests:
    cpu:    250m
    memory: 256Mi
  limits:
    cpu:    500m
    memory: 512Mi
```

Tip

Start with requests based on real usage.
Set limits to protect the cluster.

4 BIN PACKING

- Kubernetes tries to pack Pods tightly on nodes.
- Good bin packing = better utilization.
- Over-requesting leads to wasted capacity.
- Under-requesting leads to performance issues.



HOW TO IMPROVE BIN PACKING

- ✓ Set accurate requests.
- ✓ Remove unused / idle workloads.
- ✓ Use Cluster Autoscaler.
- ✓ Use VPA to right size.
- ✓ Avoid huge differences in requests across pods.

5 COST OPTIMIZATION

- Right sizing saves cloud costs.
- Over-provisioning = wasted money.
- Use spot/preemptible nodes.
- Scale down when not needed.
- Monitor & optimize constantly.

COST SAVING IDEAS

- Right size resources
- Use HPA & VPA
- Use Cluster Autoscaler
- Use Spot / Preemptible nodes
- Remove unused resources

6 PRODUCTION TUNING CHECKLIST

- ✓ Monitor CPU & Memory usage (Prometheus/Grafana).
- ✓ Check throttling & OOM events.
- ✓ Review requests & limits regularly.
- ✓ Use VPA recommendations.
- ✓ Enable HPA for variable workloads.
- ✓ Use Cluster Autoscaler for nodes.
- ✓ Run load tests before production.
- ✓ Continuously optimize & iterate.

7 PRODUCTION TUNING TABLE

WORKLOAD TYPE	CPU REQUEST	CPU LIMIT	MEMORY REQUEST	MEMORY LIMIT	NOTES
Web App	250m - 500m	500m - 1000m	256Mi - 512Mi	512Mi - 1Gi	Burstable workload
API Service	500m - 1000m	1000m - 2000m	512Mi - 1Gi	1Gi - 2Gi	Medium traffic
Worker / Job	200m - 500m	500m - 1000m	256Mi - 512Mi	512Mi - 1Gi	Batch processing
Database	1000m+	2000m+	2Gi+	4Gi+	Stateful workloads
Cache (Redis)	250m - 500m	500m - 1000m	512Mi - 1Gi	1Gi - 2Gi	In-memory cache

8 KEY METRICS TO MONITOR

- CPU Usage % (avg / max)
- Memory Usage % (avg / max)
- CPU Throttling (throttled seconds)
- OOM Kills (count)
- Pod Restarts (count)
- Node Utilization (cpu / memory)

"Optimize today. Scale tomorrow. Save forever."

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES PRODUCTION ENGINEERING

14. KUBERNETES LOGGING

"You can't fix what you can't see."

1 CONTAINER LOGS

- Every container writes logs to stdout and stderr.
- Docker / container runtime redirects logs to log files.
- Default location on the node: /var/log/containers/

HOW IT WORKS

Container → stdout/stderr logs → Node File

File: /var/log/containers/<pod>_<ns>_<container>-<id>.log

TIP: Always design apps to log to stdout/stderr.

2 KUBELET LOGS

- Kubelet manages Pods, containers and communicates with API Server.
- Logs are useful for troubleshooting Pod startup, readiness & lifecycle.
- Location: /var/log/kubelet.log
- View with: journalctl -u kubelet

KEY INFO

- Pod lifecycle events
- Image pulls
- Container status
- Probe results
- Node registration

3 FLUENT BIT

- Lightweight, high performance log processor and forwarder.
- Written in C, uses low resources.
- Collects logs from files, Docker, Kubernetes, systemd, etc.
- Forwards to multiple destinations.
- Best for DaemonSet on nodes.

FEATURES

- ✓ Low CPU & memory usage
- ✓ Multi-source input
- ✓ Multi-destination output
- ✓ Filtering & parsing
- ✓ Supports buffers & retries
- ✓ Perfect for edge / nodes

4 FLUENTD

- Open source data collector.
- More plugins and features than Fluent Bit.
- Written in Ruby.
- Good for complex routing & transformations.
- Often used in larger setups.

FEATURES

- ✓ Rich plugin ecosystem
- ✓ Advanced routing
- ✓ Data transformation
- ✓ Buffering & reliability
- ✓ Output to many systems

fluentd

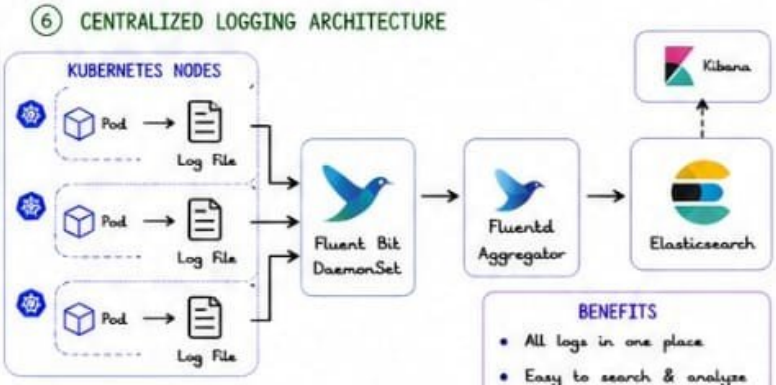
5 ELASTICSEARCH

- Distributed search & analytics engine.
- Stores and indexes logs for fast search.
- Scalable, high availability, real-time.
- Works great with Kibana for visualization.
- Part of the ELK Stack.

WHY ELASTICSEARCH?

- ✓ Fast full-text search
- ✓ Real-time indexing
- ✓ Scalable & resilient
- ✓ Powerful aggregations
- ✓ Wide community support

6 CENTRALIZED LOGGING ARCHITECTURE



BENEFITS

- All logs in one place
- Easy to search & analyze
- Real-time monitoring
- Alerting & dashboards

7 LOG FLOW (END TO END)

Application writes logs → Container stdout/stderr → Node Log File → Fluent Bit (DaemonSet) → Fluentd (Aggregator) → Elasticsearch (Store/Index) → Kibana (Visualize)

8 IMPORTANT PATHS & COMMANDS

PATHS

Container logs:	/var/log/containers/
Kubelet logs:	/var/log/kubelet.log
Docker logs:	/var/log/docker.log
System logs:	/var/log/syslog
Journal logs:	journalctl -u kubelet

USEFUL COMMANDS

```
# View pod logs
kubectl logs <pod-name> -n <namespace>
# View previous container logs
kubectl logs <pod-name> -c <container> --previous
# Follow logs
kubectl logs -f <pod-name> -n <namespace>
# Describe pod (events)
kubectl describe pod <pod-name> -n <namespace>
```

9 BEST PRACTICES

- ✓ Write logs to stdout/stderr.
- ✓ Use structured logging (JSON) for better parsing.
- ✓ Include timestamps, log levels, request IDs.
- ✓ Use Fluent Bit for node level log collection.
- ✓ Store logs in Elasticsearch and visualize in Kibana.
- ✓ Set up log retention & rotation policies.
- ✓ Monitor your logging pipeline.

GOOD LOGS

- ✓ Structured
- ✓ Meaningful
- ✓ Consistent
- ✓ Searchable

10 PRODUCTION EXAMPLES

E-COMMERCE APP
- Track orders
- Debug payments
- Monitor errors

MICROSERVICES
- Trace requests
- Service errors
- Latency logs

SECURITY
- Audit logs
- Login attempts
- Threat detection

OBSERVABILITY
- Monitor behavior
- SLA tracking
- Alerts & insights

COMPLIANCE
- Retain logs
- Meet standards
- Audit ready

"Good logging is the foundation of great observability, reliability, and security."

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES PRODUCTION ENGINEERING

15. KUBERNETES MONITORING

"Observe everything. Alert early. Stay ahead."

1 WHAT IS MONITORING?

- Monitoring helps you understand what is happening in your cluster.
- Collects metrics, stores, visualizes and alerts.
- Essential for performance, availability and troubleshooting.

MONITORING GOALS

- ✓ Know the health of the cluster
- ✓ Detect issues early
- ✓ Understand performance
- ✓ Plan capacity
- ✓ Ensure reliability

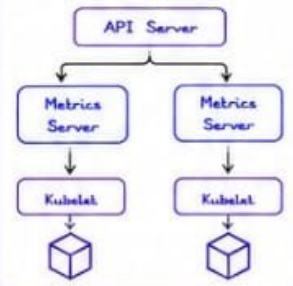
2 METRICS SERVER

- Collects resource metrics from Kubelet.
- Used by HPA, kubectl top, VPA.
- Provides CPU & Memory usage.
- Not a long-term storage solution.

INSTALL METRICS SERVER

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/
releases/latest/download/components.yaml
```

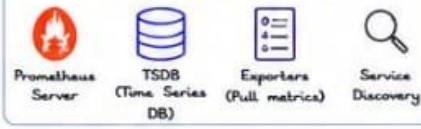
HOW IT WORKS



3 PROMETHEUS

- Powerful open-source monitoring system.
- Pulls metrics from configured targets.
- Stores time-series data locally.
- Uses PromQL for querying.

PROMETHEUS COMPONENTS



KEY FEATURES

- ✓ Multi-dimensional data model.
- ✓ Powerful query language (PromQL).
- ✓ Great alerting support.
- ✓ Highly scalable.
- ✓ Multiple exporters support.

Prometheus

4 GRAFANA

- Visualization and analytics platform.
- Connects to Prometheus and other data sources.
- Create dashboards and explore metrics.
- Supports alerting and annotations.

GRAFANA FEATURES

POPULAR DASHBOARD USE CASES

- ✓ Cluster overview
- ✓ Node performance
- ✓ Pod performance
- ✓ Application metrics
- ✓ K8s component health

5 KUBE-STATE-METRICS

- Generates metrics about Kubernetes objects.
- Deployments, Pods, Nodes, Jobs, etc.
- Does not collect resource usage.
- Essential for monitoring cluster state.

EXPOSES METRICS FOR

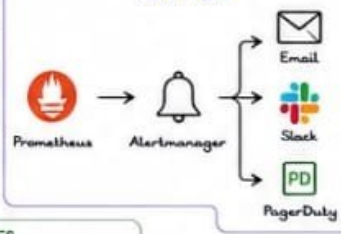
- ✓ Deployments & Replicas
- ✓ Pods status
- ✓ Nodes
- ✓ Jobs & CronJobs
- ✓ ConfigMaps, Secrets, etc.

kube-state-metrics

6 ALERTMANAGER

- Handles alerts sent by Prometheus.
- Deduplicates, groups and routes alerts.
- Integrates with email, Slack, PagerDuty, etc.
- Ensures alerts reach the right people.

ALERT FLOW



ALERTMANAGER FEATURES

- ✓ Alert grouping & deduplication
- ✓ Silencing & inhibition
- ✓ Multiple notification channels
- ✓ High availability setup

7 MONITORING ARCHITECTURE

8 COMPONENT OVERVIEW

COMPONENT	PURPOSE	COLLECTS	STORES	VISUALIZES	ALERTS
Metrics Server	Resource metrics	CPU, Memory	No	No	No
Prometheus	Metrics collection & storage	All metrics	Yes (TSDB)	No	Yes (→ Alertmanager)
kube-state-metrics	K8s object metrics	Cluster state	No	No	No
Grafana	Visualization	From Prometheus	No	Yes	Yes (optional)
Alertmanager	Alert handling	Alerts	No	No	Yes

9 BEST PRACTICES

- ✓ Monitor everything important.
- ✓ Use meaningful dashboards.
- ✓ Set alerts based on SLOs.
- ✓ Avoid alert fatigue.
- ✓ Test alerts regularly.
- ✓ Keep retention based on needs.

Tip
Good monitoring saves you when things go wrong. You can't improve what you don't measure.

10 COMMON EXPORTERS

11 KEY METRICS TO MONITOR

- ✓ CPU usage %
- ✓ Memory usage %
- ✓ Pod restarts
- ✓ Node disk pressure
- ✓ Request latency
- ✓ Error rate
- ✓ Queue length
- ✓ Cluster component health

"Monitor the unknown. Alert the important. Visualize the truth."

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES PRODUCTION ENGINEERING

16. PRODUCTION TROUBLESHOOTING

"Find the root cause. Fix the issue. Prevent it next time."

1 CRASHLOOPBACKOFF

Pod starts, crashes, and Kubernetes keeps restarting it.

COMMON CAUSES
- Application error or bug
- Missing configuration
- Wrong command/args
- Failed dependencies
- Insufficient resources

TROUBLESHOOTING STEPS
```
# Check pod status
kubectl get pods <pod>
# Describe pod
kubectl describe pod <pod>
# Check logs
kubectl logs <pod> -p
# Check events
kubectl get events --sort-by=.lastTimestamp
```

FIX IDEAS
- ✓ Check logs for errors.
- ✓ Validate configs, secrets, env.
- ✓ Fix code or command.
- ✓ Add health checks & resource limits.

2 IMAGEPULLBACKOFF

Kubernetes can't pull the container image.

COMMON CAUSES
- Wrong image name/tag
- Image doesn't exist
- Private registry auth issue
- Network/DNS issues
- Rate limiting (Docker Hub)

TROUBLESHOOTING STEPS
```
# Describe pod
kubectl describe pod <pod>
# Check events
kubectl get events --sort-by=.lastTimestamp
# Manually pull image (on node)
docker pull <image:tag>
# Check imagePullSecrets
kubectl get secrets
```

FIX IDEAS
- ✓ Verify image name & tag.
- ✓ Check registry credentials.
- ✓ Create/attach imagePullSecrets.
- ✓ Check network & DNS.

3 PENDING PODS

Pod is not scheduled onto any node.

COMMON CAUSES
- Insufficient resources
- Node selector mismatch
- Taints/Tolerations
- PVC not bound
- Scheduling constraints

TROUBLESHOOTING STEPS
```
# Check pod
kubectl describe pod <pod>
# Check nodes
kubectl get nodes
# Check events
kubectl get events --field-selector
involvedObject.name=<pod>
```

CHECK THIS
- ✓ CPU/Memory availability
- ✓ Node taints
- ✓ Node selectors
- ✓ PVC status
- ✓ Affinity rules

FIX IDEAS
- ✓ Add/scale nodes.
- ✓ Fix requests/limits.
- ✓ Adjust node selector / tolerations.
- ✓ Ensure PVC is bound.
- ✓ Review affinity & constraints.

4 FAILED SCHEDULING

Scheduler can't place the pod on any node.

COMMON CAUSES
- Not enough CPU/Memory
- No matching nodes
- Node taints
- Too many pods on nodes
- Pod anti-affinity rules

TROUBLESHOOTING STEPS
```
# Describe pod
kubectl describe pod <pod>
# Check nodes
kubectl get nodes -o wide
# Check resource usage
kubectl top nodes
# Check events
kubectl get events --sort-by=.lastTimestamp
```

FIX IDEAS
- ✓ Add more resources or nodes.
- ✓ Relax constraints/affinity if needed.
- ✓ Use Cluster Autoscaler.
- ✓ Spread pods with topologySpreadConstraints.

5 OOMKILLED

Container killed by Linux OOM killer (out of memory).

COMMON CAUSES
- Memory limit too low
- Memory leak in app
- Spike in memory usage
- Too many pods on node

HOW TO CONFIRM
```
# Describe pod
kubectl describe pod <pod>
```
Look for:
State: Terminated
Reason: OOMKilled

FIX IDEAS
- ✓ Increase memory limit.
- ✓ Optimize application memory.
- ✓ Add Vertical Pod Autoscaler (VPA).
- ✓ Monitor memory usage.

6 TROUBLESHOOTING WORKFLOW

A systematic approach to solve issues faster.

1. **Observe** — What's wrong? Check alerts, logs, events.
2. **Identify** — Identify the failing component (Pod/Node/App).
3. **Investigate** — Use kubectl commands & logs.
4. **Fix** — Apply the fix (config, code, resources).
5. **Verify** — Confirm the issue is resolved.

USEFUL KUBECTL COMMANDS
- kubectl get pods -A
- kubectl describe pod <pod>
- kubectl logs <pod> -p
- kubectl get events --sort-by=.lastTimestamp
- kubectl top pods -A
- kubectl top nodes
- kubectl get nodes -o wide
- kubectl explain <resource>

7 COMMON POD STATUS QUICK REFERENCE

STATUS	MEANING	WHAT TO DO
Running	Pod is running fine	Monitor
Pending	Pod not scheduled yet	Check events & scheduler
CrashLoopBackOff	Pod crashes repeatedly	Check logs (-p) & fix app
ImagePullBackOff	Cannot pull image	Check image & secrets
OOMKilled	Killed due to OOM	Increase memory limit
Completed	Job/Pod completed	Check logs if needed
Unknown	Pod status unknown	Check node & network

TIP

Always check Events first. They tell you what Kubernetes is trying to say.

8 PREVENTION BEST PRACTICES

- ✓ Set proper requests & limits.
- ✓ Use liveness, readiness & startup probes.
- ✓ Monitor resources & set alerts.
- ✓ Use resource quotas & limit ranges.
- ✓ Keep images updated & scanned.
- ✓ Test deployments in staging.
- ✓ Enable autoscaling.
- ✓ Document runbooks & incident steps.

9 TOOLS THAT HELP

- kubectl
- Lens
- k9s
- Prometheus
- Grafana

"Problems will happen. Proactive teams fix fast and learn faster."

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES PRODUCTION ENGINEERING

17. INCIDENT RESPONSE

"Be prepared. Act fast. Minimize impact. Learn always."

1 INCIDENT DETECTION

- Detect issues early using monitoring and alerting.
- Alerts from Prometheus, Grafana, Alertmanager, or logs.
- Indicators: High error rate, latency, CPU, memory, failed pods, user reports.
- Validate the alert before escalating.

DETECTION SOURCES

- Prometheus Alerts
- Grafana Dashboards
- Alertmanager Notifications
- Application Logs
- User Reports
- Synthetic Monitoring

2 CONTAINMENT

- Stop the bleeding and limit impact.
- Isolate affected components.
- Apply temporary fixes or rollbacks.
- Protect the rest of the system.
- Communicate the impact.

CONTAINMENT ACTIONS

- ✓ Scale down / cordon nodes
- ✓ Disable traffic (Ingress / LB)
- ✓ Rollback / Revert change
- ✓ Kill faulty pods
- ✓ Apply security rules
- ✓ Limit blast radius

CONTAINMENT FLOW

Alert Triggered → Confirm Impact → Isolate Component → Communicate Status → Stabilize System

3 INVESTIGATION

- Find out what happened and why.
- Collect logs, metrics, events and traces.
- Compare recent changes.
- Reproduce if possible.
- Gather facts, not assumptions.

INVESTIGATION CHECKLIST

- ✓ Review logs & events
- ✓ Check metrics & dashboards
- ✓ Inspect pod status & events
- ✓ Review recent deployments
- ✓ Check config changes
- ✓ Look at network & DNS
- ✓ Identify affected services

USEFUL KUBECTL COMMANDS

```
$ kubectl get pods -A
$ kubectl describe pod <pod-name>
$ kubectl logs <pod-name> -p
$ kubectl get events --sort-by=.lastTimestamp
$ kubectl top pods -A
$ kubectl get nodes
$ kubectl describe node <node-name>
$ kubectl get svc,ing,cm,secret -A
```

DATA TO COLLECT

- ✓ Logs
- ✓ Metrics
- ✓ Events
- ✓ Traces
- ✓ Configs
- ✓ Change history
- ✓ User impact

4 RECOVERY

- Restore normal service.
- Apply the fix or permanent solution.
- Verify the system is healthy.
- Monitor closely after recovery.
- Update runbooks if needed.

RECOVERY ACTIONS

- ✓ Apply fix / configuration
- ✓ Redeploy fixed version
- ✓ Scale back up
- ✓ Verify system health
- ✓ Monitor for regression
- ✓ Close the incident

HEALTH VERIFICATION

Metrics Normal | Pods Running | Services Responding | Users Happy

Tip

Recover in small steps. Validate after each step. Don't assume it's fixed until metrics agree.

5 ROOT CAUSE ANALYSIS (RCA)

- Identify the real root cause.
- Understand why it happened.
- Identify gaps in process or system.
- Prevent recurrence.
- Document learnings.

RCA METHODS

- 5 Whys
- Timeline Analysis
- Fishbone Diagram
- Fault Tree Analysis

EXAMPLE: 5 WHYS

Problem:	API is down	
Why 1:	Why is API down?	→ Pod crashed
Why 2:	Why did pod crash?	→ OOMKilled
Why 3:	Why OOMKilled?	→ High memory usage
Why 4:	Why high memory usage?	→ Memory leak
Why 5:	Why memory leak?	→ Bug in code

Root Cause: Memory leak in application code

RCA OUTPUT

- ✓ Root cause identified
- ✓ Contributing factors
- ✓ Action items
- ✓ Preventive measures
- ✓ Runbook updates

6 COMMUNICATION

- Communicate clearly and frequently.
- Update stakeholders on status.
- Be honest and transparent.
- Share ETA and impact.
- Documunicate resolution and lessons learned.

COMMUNICATION PLAN

Who to Notify?	What to Share?	How?	When?	Tone
On-call, Team Leads, Mgmt, Customers	Impact, Status, Actions, ETA, Resolution	Slack, Email, PagerDuty, MS Teams, Status Page	Immediately, Regular updates, After resolution	Calm, Clear, Empathetic, Professional

COMMUNICATION EXAMPLE

Incident: API latency high
Impact: Users may experience slow responses
Status: Investigating
Next Update: 20 mins
ETA: 1 hour
We'll keep you updated.

INCIDENT SEVERITY LEVELS

SEVERITY	IMPACT	RESPONSE TIME	EXAMPLE
P1 - Critical	System down, major impact	0 - 15 min	Outage, Data loss
P2 - High	Degraded service	15 - 60 min	Major feature down
P3 - Medium	Minor impact	1 - 4 hours	Slow performance
P4 - Low	Minimal / No impact	4+ hours	Cosmetic issue

POST-INCIDENT REVIEW

- ✓ What happened?
- ✓ What went well?
- ✓ What didn't?
- ✓ What will we improve?
- ✓ Update runbooks & docs

TOOLS THAT HELP

Prometheus | Grafana | Loki | ELK Stack | Kubernetes | PagerDuty | Slack | Git

"Great teams don't avoid incidents. They prepare, respond, learn, and get better."

VERIQTA

VERIQTA

@ veriqta

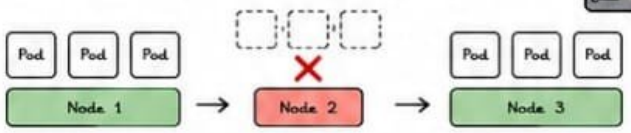
VOLUME 2
KUBERNETES PRODUCTION ENGINEERING

18. PRODUCTION CASE STUDIES

"Learn from real failures. Prevent future outages."

1 NODE FAILURE

Scenario: A worker node went down unexpectedly.



ROOT CAUSE	IMPACT	RESOLUTION	LESSONS LEARNED
Hardware failure on the node.	• Pods on the node evicted • Service latency increased.	✓ K8s rescheduled pods to healthy nodes. ✓ Node replaced.	✓ Use multi-node clusters ✓ Monitor node health ✓ Enable auto-scaling ✓ Have spare capacity

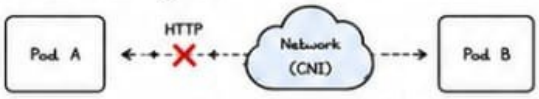
2 STORAGE FAILURE

Scenario: Persistent Volume became unresponsive.

ROOT CAUSE	IMPACT	RESOLUTION	LESSONS LEARNED
Disk/Volume corruption in the storage backend.	• Applications crashed. • Data access failed.	✓ Failover to backup storage. ✓ Volume restored from snapshot.	✓ Take regular snapshots ✓ Test backups ✓ Use resilient storage ✓ Monitor disk health

3 NETWORK OUTAGE

Scenario: Network plugin misconfiguration caused a cluster-wide connectivity issue.



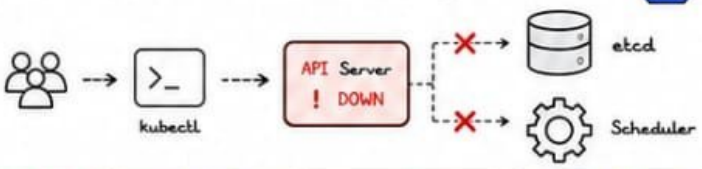
ROOT CAUSE	IMPACT	RESOLUTION	LESSONS LEARNED
CNI plugin config error blocked pod-to-pod traffic.	• Services unreachable • Health checks failed • Users impacted	✓ Fixed CNI config ✓ Restarted plugin pods ✓ Connectivity restored.	✓ Validate CNI changes before rollout ✓ Have network policies documented ✓ Monitor network metrics

CHECKLIST

- ✓ Network Policies
- ✓ CNI Health
- ✓ DNS Resolution
- ✓ Service Connectivity
- ✓ Egress Testing

4 API SERVER OUTAGE

Scenario: API Server became unavailable due to high load.



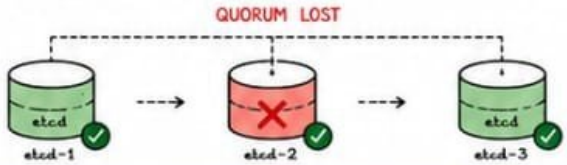
ROOT CAUSE	IMPACT	RESOLUTION	LESSONS LEARNED
High load and insufficient resources on API Server.	• kubectl commands failed • New deployments blocked • System-wide impact	✓ Scaled control plane resources ✓ Restarted API Server ✓ Load normalized.	✓ Monitor API Server metrics ✓ Set resource limits ✓ Use HA control plane ✓ Plan capacity

MONITOR THESE METRICS

- API Server Latency
- Request Rate
- Error Rate
- etcd Latency
- Active Connections

5 ETCD FAILURE

Scenario: etcd cluster lost quorum.



ROOT CAUSE	IMPACT	RESOLUTION	LESSONS LEARNED
etcd member failed causing loss of quorum.	• API Server read failures • Writes blocked • Cluster unstable	✓ Restored etcd member from backup ✓ Rejoined cluster ✓ Quorum restored.	✓ Run odd number of etcd nodes (3,5,7) ✓ Backup etcd regularly ✓ Monitor etcd health

ETCD HEALTH CHECKS

- etcdctl endpoint health
- Snapshot Status
- DB Size Growth
- Leader Election
- Disk Usage

LESSONS LEARNED ACROSS INCIDENTS

- ✓ Redundancy saves the day.
- ✓ Monitoring is your early warning.
- ✓ Automate recovery when possible.
- ✓ Test disaster recovery regularly.
- ✓ Document everything.
- ✓ Communicate clearly & quickly.
- ✓ Learn, improve, and prevent repeat.

GOLDEN RULE

You don't rise to the level of your goals. You fall to the level of your systems.

POST-INCIDENT REVIEW TEMPLATE

- What Happened?
- Impact Summary
- Root Cause
- What Went Well?
- What Didn't Go Well?
- Action Items
- Owner & Due Date

Document → Share → Improve → Repeat

"Every outage is a lesson. Every lesson makes your systems stronger."

VERIQTA @ veriqta

VOLUME 2 KUBERNETES PRODUCTION ENGINEERING

19. KUBERNETES PRODUCTION CHEAT SHEET

"Keep it simple. Automate everything. Monitor always."

1 KUBECTL PRODUCTION COMMANDS

TASK	COMMANDS
Get all pods (all namespaces)	`kubectl get pods -A`
Get pods wide	`kubectl get pods -A -o wide`
Describe a pod	`kubectl describe pod <pod> -n <ns>`
Get deployments	`kubectl get deploy -A`
Describe deployment	`kubectl describe deploy <deploy> -n <ns>`
Get services	`kubectl get svc -A`
Get nodes	`kubectl get nodes -o wide`
Get events	`kubectl get events -A --sort-by=.lastTimestamp`
Get all resources in ns	`kubectl get all -n <ns>`
Delete a resource	`kubectl delete <type> <name> -n <ns>`
Apply manifest	`kubectl apply -f file.yaml`
Edit resource	`kubectl edit <type> <name> -n <ns>`

Tip: Always use `-n <namespace>` in production!

2 SCALING COMMANDS

TASK	COMMAND
Scale deployment	`kubectl scale deploy <name> --replicas=5 -n <ns>`
Scale statefulset	`kubectl scale sts <name> --replicas=3 -n <ns>`
Scale daemonset	`kubectl scale ds <name> --replicas=2 -n <ns>`
Get HPA	`kubectl get hpa -A`
Create HPA	`kubectl autoscale deploy <name> \ --cpu-percent=70 --min=2 --max=10 -n <ns>`
Edit HPA	`kubectl edit hpa <name> -n <ns>`
Delete HPA	`kubectl delete hpa <name> -n <ns>`
Check resource usage	`kubectl top pods -n <ns>`
Check nodes usage	`kubectl top nodes`

3 DEBUGGING COMMANDS

TASK	COMMAND
Describe pod (detailed)	`kubectl describe pod <pod> -n <ns>`
Get pod logs	`kubectl logs <pod> -n <ns>`
Logs from previous container	`kubectl logs <pod> -c <container> \ --previous -n <ns>`
Follow logs	`kubectl logs -f <pod> -n <ns>`
Execute into pod	`kubectl exec -it <pod> -n <ns> -- /bin/sh`
Get pod shell (bash)	`kubectl exec -it <pod> -n <ns> -- /bin/bash`
Describe node	`kubectl describe node <node>`
Check node conditions	`kubectl get nodes`
Check pod status	`kubectl get pod <pod> -n <ns> -o wide`
Events for pod	`kubectl get events --field-selector \ involvedObject.name=<pod> -n <ns>`

4 MONITORING COMMANDS

TASK	COMMAND
Metrics Server check	`kubectl top nodes`
Top pods by resource	`kubectl top pods -A --sort-by=cpu`
Top pods by memory	`kubectl top pods -A --sort-by=memory`
Get HPA status	`kubectl get hpa -A -o wide`
Get resource quota	`kubectl get quota -A`
Get limit ranges	`kubectl get limits -A`
Get cluster info	`kubectl cluster-info`
Get component status	`kubectl get --raw /readyz`
Get API resources	`kubectl api-resources`
Check API latency	`kubectl get --raw /metrics | \ grep apiserver_request_duration`

5 LOGGING COMMANDS

TASK	COMMAND
Get logs	`kubectl logs <pod> -n <ns>`
Logs with timestamps	`kubectl logs <pod> -n <ns> \ --timestamps`
All containers logs	`kubectl logs <pod> -n <ns> -c <c>`
Tail last 100 lines	`kubectl logs <pod> -n <ns> --tail=100`
Follow logs	`kubectl logs -f <pod> -n <ns>`
Logs from previous	`kubectl logs <pod> -c <c> --previous -n <ns>`
Log all pods in ns	`for p in $(kubectl get pods -n <ns> -o name); \ do echo "--- $p ---"; kubectl logs $p -n <ns>; done`
Get kubelet logs	`journalctl -u kubelet -f`
Get cluster events	`kubectl get events -A --sort-by=.lastTimestamp`

6 QUICK REFERENCE TABLES

POD STATUS MEANING

STATUS	MEANING
Running	Pod is running fine
Pending	Pod is not scheduled yet
CrashLoopBackOff	Container keeps crashing
ImagePullBackOff	Cannot pull container image
ErrImagePull	Image pull error
Completed	Job/Pod completed
OOMKilled	Killed due to memory limit
Unknown	Pod status unknown

RESTART REASONS

REASON	MEANING
OOMKilled	Out of memory
Error	Application error
CrashLoopBackOff	Container crashed repeatedly
Completed	Container finished
Evicted	Pod evicted by node
ImagePullBackOff	Image pull failed

COMMON PORTS

SERVICE	PORT
Kubernetes API	6443
Etcd	2379, 2380
Kubelet	10250
Kube-scheduler	10259
Kube-controller	10257
NodePort Range	30000-32767

RESOURCE DEFAULTS (TYPICAL)

RESOURCE	DEFAULT
CPU Request	100m
CPU Limit	500m
Memory Request	128Mi
Memory Limit	512Mi
Ephemeral Storage	1Gi

7 PRODUCTION CHECKLIST (QUICK)

- ✓ Use resource requests & limits.
- ✓ Set liveness, readiness & startup probes.
- ✓ Enable HPA for critical workloads.
- ✓ Use anti-affinity & PodDisruptionBudgets.
- ✓ Enable monitoring & alerting.
- ✓ Centralize logs.
- ✓ Backup etcd regularly.
- ✓ Test rollbacks & disaster recovery.
- ✓ Use namespaces & RBAC.
- ✓ Scan images & use trusted registries.
- ✓ Keep cluster, nodes & add-ons updated.

REMEMBER

A healthy cluster is not about luck. It's about preparation, observability, and automation.

8 ONE-LINER HERO COMMANDS

All pods not Running

```
kubectl get pods -A \
--field-selector \
status.phase!=Running
```

Pods by Node

```
kubectl get pods -A \
-o wide --sort-by=.spec.
nodeName
```

Delete pods by label

```
kubectl delete pods -l
app=myapp -n <ns>
```

Drain node safely

```
kubectl drain <node> \
--ignore-daemonsets \
--delete-emptydir-data
```

Cordon node

```
kubectl cordon <node>
```

Uncordon node

```
kubectl uncordon <node>
```

Get YAML of resource

```
kubectl get deploy <name> \
-n <ns> -o yaml
```

Port forward

```
kubectl port-forward svc/<svc> \
8080:80 -n <ns>
```

"Know your cluster. Trust your tools. Automate your workflows. Ship with confidence."

VERIQTA

VERIQTA

@ veriqta

VOLUME 2
KUBERNETES PRODUCTION ENGINEERING

20. KUBERNETES PRODUCTION READINESS CHECKLIST

"Prepare today. Protect tomorrow. Deliver always."

1 CLUSTER HEALTH CHECK

- [ ] All nodes are Ready
- [ ] No node under Memory/Disk pressure
- [ ] No excessive Pod restarts
- [ ] All system pods are Running
- [ ] kubelet and kube-proxy healthy
- [ ] API Server, Controller Manager, Scheduler healthy
- [ ] etcd cluster healthy and synced
- [ ] Sufficient resources (CPU, Memory, Disk)
- [ ] Cluster autoscaler configured (if needed)

QUICK CHECK

```
kubectl get nodes
kubectl get pods -A
kubectl top nodes
kubectl top pods -A
kubectl get componentstatuses
```

2 SECURITY REVIEW

- [ ] RBAC is enabled and least privilege enforced
- [ ] No default service accounts in use
- [ ] Pod Security Standards/Policies enforced
- [ ] NetworkPolicies defined and applied
- [ ] Secrets are encrypted at rest
- [ ] Image scanning enabled
- [ ] Admission controllers enabled
- [ ] Audit logging enabled
- [ ] No privileged containers
- [ ] Regular security updates applied

QUICK CHECK

```
kubectl get clusterrole
kubectl get networkpolicies -A
kubectl get secrets -A
kubectl get podsecuritypolicies
(or psp replacement)
kubectl get events -A
```

3 SCALING REVIEW

- [ ] Deployments have resource requests & limits
- [ ] HPA configured for critical workloads
- [ ] Cluster Autoscaler tested and working
- [ ] Vertical Pod Autoscaler considered
- [ ] Node groups sized appropriately
- [ ] Pod disruption budgets (PDB) configured
- [ ] Load balancer configured (if external)
- [ ] Autoscaling alerts configured

QUICK CHECK

```
kubectl get hpa
kubectl get pdb
kubectl get clusterscaler
kubectl describe hpa <name>
```

4 MONITORING REVIEW

- [ ] Metrics Server is healthy
- [ ] Prometheus collecting metrics
- [ ] Grafana dashboards configured
- [ ] Alertmanager configured
- [ ] Key alerts created & tested
- [ ] Node, Pod, API metrics visible
- [ ] Blackbox / uptime monitoring in place
- [ ] SLOs / SLIs defined (if applicable)

QUICK CHECK

```
kubectl top nodes
kubectl top pods -A
curl -s http://prometheus:9090/-/healthy
curl -s http://alertmanager:9093/-/healthy
```

5 LOGGING REVIEW

- [ ] Container logs being collected
- [ ] Fluent Bit/Fluentd running
- [ ] Logs shipped to central storage
- [ ] Log retention policy configured
- [ ] Important log alerts configured
- [ ] Structured logging enabled
- [ ] Log access controlled & secure

QUICK CHECK

```
kubectl logs <pod>
kubectl logs <pod> -n <ns>
kubectl get ds -n logging
Check in ELK / Loki /OpenSearch
```

6 BACKUP REVIEW

- [ ] etcd backups automated
- [ ] Backup stored off-cluster
- [ ] Application data backups configured
- [ ] Backup encryption enabled
- [ ] Backup restore tested successfully
- [ ] Backup schedule documented
- [ ] Retention policy defined

QUICK CHECK

```
etcdctl snapshot status
List backup storage location
Restore test in staging
```

7 DISASTER RECOVERY REVIEW

- [ ] DR plan documented
- [ ] Multi-zone / Multi-region strategy defined
- [ ] Infrastructure as Code (IaC) available
- [ ] Regular DR drills conducted
- [ ] RTO & RPO defined and achievable
- [ ] Critical workloads can be restored
- [ ] DNS / Traffic failover tested
- [ ] Communication plan ready

QUICK CHECK

```
Validate DR environment
Run failover test
Check DNS / LB failover
Review runbooks
```

8 FINAL PRODUCTION READINESS ASSESSMENT

- [ ] Cluster is stable
- [ ] Secure by design
- [ ] Scalable and resilient
- [ ] Observable and alerted
- [ ] Logs centralized
- [ ] Backups proven
- [ ] DR Tested
- [ ] Runbooks ready
- [ ] Team trained
- [ ] Go for Production

READINESS SCORECARD

AREA	STATUS	SCORE
Cluster Health	☐ Pass ☐ Fail	__/10
Security	☐ Pass ☐ Fail	__/10
Scaling	☐ Pass ☐ Fail	__/10
Monitoring	☐ Pass ☐ Fail	__/10
Logging	☐ Pass ☐ Fail	__/10
Backup	☐ Pass ☐ Fail	__/10
Disaster Recovery	☐ Pass ☐ Fail	__/10
TOTAL SCORE		__/70

PRODUCTION GOLDEN RULES

- Automate everything possible.
- Monitor everything important.
- Secure by default, not by option.
- Test failure before failure happens.
- Document everything clearly.
- Communicate clearly and quickly.
- Continuously improve and learn.

TIP

A ready cluster is a reliable cluster. Reliability builds trust.

"A little preparation prevents a lot of outages. Be ready. Stay ready."